

 Latest updates: <https://dl.acm.org/doi/10.1145/3769811>

Published: 05 December 2025

RESEARCH-ARTICLE

[Citation in BibTeX format](#)

P2FedRec: Towards Privacy-Preserving and Personalized Federated Recommendation via Relationship Awareness

CHENFEI HU, Beijing Institute of Technology, Beijing, China

ZIHAI XU, Changchun University, Changchun, Jilin, China

TONG WU, University of Science and Technology Beijing, Beijing, China

YOU LI, Beijing Institute of Technology, Beijing, China

CHUAN ZHANG, Beijing Institute of Technology, Beijing, China

LIEHUANG ZHU, Beijing Institute of Technology, Beijing, China

Open Access Support provided by:

Beijing Institute of Technology

University of Science and Technology Beijing

Changchun University

P²FedRec: Towards Privacy-Preserving and Personalized Federated Recommendation via Relationship Awareness

CHENFEI HU, Beijing Institute of Technology, China

ZIHAO XU, Changchun University, China

TONG WU, University of Science and Technology Beijing, China

YOU LI, Beijing Institute of Technology, China

CHUAN ZHANG*, Beijing Institute of Technology, China

LIEHUANG ZHU, Beijing Institute of Technology, China

Personalized federated recommendation systems can not only extract common prior knowledge from extensive decentralized data but also provide personalized models for different users to achieve independent and customized services. Incorporating user relationship graphs to enhance personalized modeling is highly promising in federated recommendation. However, it is challenging to construct such graphs and further capture personalized user information while guaranteeing multi-level (i.e., data-level and edge-level) privacy in reality. To this end, in this paper, we propose **P²FedRec**, a relationship-aware **Privacy-preserving and Personalized Federated Recommendation** scheme, which can achieve multi-level privacy protection with personalized modeling guarantees. Specifically, we first develop a user-server collaborative mechanism for relationship graph generation and user-specific preferences capture in a privacy-preserving manner. Then, we design an embedding-shared local graph construction module and a noisy global graph-guided aggregation module to safeguard the data-level and edge-level privacy, respectively. Moreover, we introduce a personalized model training module that enables users to learn tailored local models. Theoretical analysis demonstrates that P²FedRec achieves both data-level and edge-level privacy preservation on the user and server sides. Extensive experiments conducted on five real-world datasets highlight the outstanding performance of P²FedRec.

CCS Concepts: • **Security and privacy** → **Privacy-preserving protocols**; *Distributed systems security*.

Additional Key Words and Phrases: Federated Learning; Recommendation Systems; Privacy Preservation

ACM Reference Format:

Chenfei Hu, Zihao Xu, Tong Wu, You Li, Chuan Zhang, and Liehuang Zhu. 2025. P²FedRec: Towards Privacy-Preserving and Personalized Federated Recommendation via Relationship Awareness. *Proc. ACM Manag. Data* 3, 6 (SIGMOD), Article 346 (December 2025), 25 pages. <https://doi.org/10.1145/3769811>

1 Introduction

Recently, recommendation systems have been widely adopted across various domains such as streaming media [5], e-commerce [39], and social networks [45] due to their capability of delivering

*Chuan Zhang is the corresponding author.

Authors' Contact Information: Chenfei Hu, Beijing Institute of Technology, Beijing, China, chenfeih@bit.edu.cn; Zihao Xu, Changchun University, Changchun, China, 230702293@mails.ccu.edu.cn; Tong Wu, University of Science and Technology Beijing, Beijing, China, tongw@ustb.edu.cn; You Li, Beijing Institute of Technology, Beijing, China, liyou_1114@163.com; Chuan Zhang, Beijing Institute of Technology, Beijing, China, chuanz@bit.edu.cn; Liehuang Zhu, Beijing Institute of Technology, Beijing, China, liehuangz@bit.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2836-6573/2025/12-ART346

<https://doi.org/10.1145/3769811>

personalized content with high precision. Despite their effectiveness, these systems face growing privacy concerns as they typically rely on centralized servers to aggregate historical user behavior data for model training [19], creating inherent risks in data collection practices. Federated learning, as a distributed machine learning paradigm, enables collaborative model training without direct data sharing. This unique capability positions it as an ideal approach for privacy-aware recommendation systems, spurring the development of federated recommendation (*FedRec*) systems.

1.1 Motivation

Although many achievements have been made, most existing FedRec systems [17, 31, 41, 49] operate under a paradigm where users collaboratively train a generic global recommendation model. Specifically, as shown in Fig. 1 (a), in each round, all users upload their locally trained models to the server, which then directly aggregates these parameters into a unified generic model and redistributes it back to all users. This one-model-fits-all approach fundamentally conflicts with the heterogeneous nature of user preferences and behavior patterns in real-world recommendation scenarios [11, 33, 42], thus creating a compelling need for personalized FedRec systems [22, 26, 27, 46] that can accommodate individual characteristics. However, these systems [22, 26, 27, 46] often treat each user as an isolated individual, overlooking the potential relationships among them. In fact, in recommendation scenarios, there are usually a variety of connections between users. For instance, on short-video platforms, users who have liked or commented on the same clip tend to share common interests and are subsequently more likely to follow similar creators or engage with related content topics; on e-commerce platforms, users who have purchased the same products tend to exhibit similar preferences and are thus more likely to purchase the same or similar products in the future.

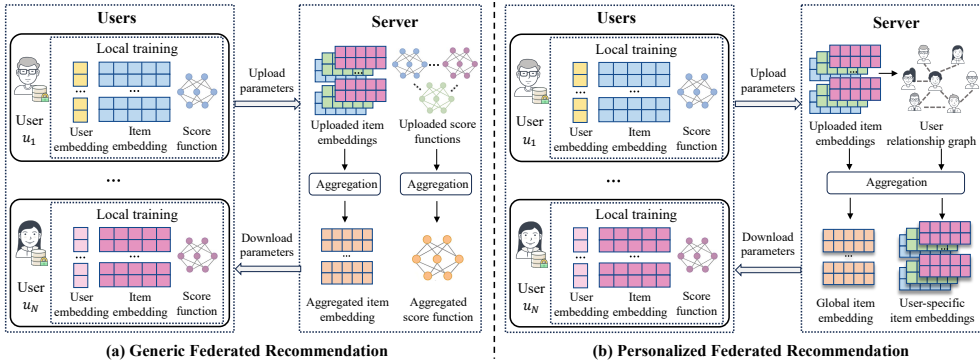


Fig. 1. Generic federated recommendation v.s. relationship-aware personalized federated recommendation.

Thus, inspired by [47], we explore a relationship-aware personalized FedRec system, depicted in Fig. 1 (b), which use a graph structure [20, 44] to capture potential relationships among users. Since the model parameters received by the server inherently capture user preferences and behavioral patterns, making them useful knowledge for constructing the relationship graph. Then, for each user, the server can use this graph to aggregate information of their neighbors to generate user-specific model parameters. These parameters are then distributed to corresponding users for local model adaptation. Although [47] has demonstrated the effectiveness of this relationship-aware training paradigm, two critical challenges remain:

- **Data-level privacy.** Although users upload only model parameters (i.e., item embeddings as shown in Fig. 1 (b)) rather than raw interaction data to the server, recent research [6]

has demonstrated that adversaries can still infer interaction data from these parameters, indicating persistent risks of data-level privacy leakage. Therefore, preserving the privacy of item embeddings is essential for ensuring data-level privacy.

- **Edge-level privacy.** In [47], the relationship graph is constructed by the server, which directly compromises the relationship (i.e., edge-level) privacy among users. The edge-level privacy is equally critical as data-level privacy, because once the historical interaction data of specific users is known, this graph can be leveraged to infer sensitive information about their neighbors. For instance, when specific users consistently interact with sensitive products (e.g., medical items), their adjacency relationships in the graph may enable the server to deduce neighbors' private attributes like health conditions or personal preferences. More critically, when such patterns are combined with overlapping geographical features, the server might further deduce professional relationships or even intimate partnerships.

1.2 Our Contributions

In light of these challenges, we propose P²FedRec, a privacy-preserving and personalized federated recommendation scheme via relationship awareness. In particular, we first develop a user-server collaborative mechanism for privacy-preserving relationship graph generation and user-specific embedding aggregation, and then design two key components to respectively guarantee data-level and edge-level privacy, namely *embedding-shared local graph construction* and *noisy global graph-guided aggregation*. Finally, follow [47], we introduce a personalized model training module that enables users to learn tailored local models.

We start by presenting the user-server collaborative mechanism used to achieve privacy-preserving relationship graph generation and user-specific embedding aggregation. This mechanism allows users to upload privacy-preserved item embeddings to the server, which calculates pairwise embedding similarities without knowing raw embeddings and distributes the results back to users. Users then recover similarities to construct local relationship graphs. Next, to derive user-specific item embeddings, users upload their privacy-preserved local graphs to servers, which perform embedding aggregation without knowing the actual graphs.

To protect the privacy of item embeddings, local differential privacy (LDP) is commonly adopted due to its high efficiency [8, 43, 46]. However, for high-dimensional embeddings, the LDP noise accumulates during server-side similarity computation, causing the results to approach a random distribution. This makes it difficult for users to accurately recover similarities, posing a significant challenge in constructing relationship graphs. To address this problem, we employ homomorphic encryption (HE), which achieves privacy protection without sacrificing accuracy. However, directly using HE to encrypt embeddings would incur significant computational and communication overhead for users. Therefore, we introduce a widely used two-server model [10, 14], and design an *embedding-shared local graph construction* module, which combines HE with additive secret sharing (ASS) to enable secure and efficient relationship graph construction.

The core of *embedding-shared local graph construction* module lies in reducing user-side overhead via ASS and using HE to assist secure computations. Specifically, this module allows users to split item embeddings into two shares using ASS and upload them to two servers separately. To enable similarity computation on these shares, the two servers require the help of Beaver triples (see Sec. 2.2). Prior works typically rely on trusted third parties to generate these triples, which is an impractical assumption. Instead, we let two servers generate Beaver triples through an HE-based protocol. Since these triples are data-agnostic, we let two servers precompute them offline to further improve training efficiency. Then, with these triples, the two servers can jointly compute the similarities efficiently while revealing nothing to either party.

Unlike item embeddings, using LDP to protect the relationship can achieve highly effective privacy protection while incurring only limited utility loss. This is because the local relationship graphs users construct are inherently dense, which naturally dilutes the noise injected by LDP and reduces its impact on the overall graph structure. Thus, we use LDP to perturb local graphs and design a *noisy global graph guided aggregation* module to mitigate the impact of LDP-introduced noise further.

In summary, by employing differentiated protection strategies for different privacy requirements, P²FedRec can achieve multi-layer privacy protection while guaranteeing utility and efficiency. Our main contributions can be summarized as follows.

- We propose a relationship-aware privacy-preserving and personalized federated recommendation scheme, termed as P²FedRec, which can achieve multi-level privacy protection with enhanced personalized modeling guarantees. To the best of our knowledge, P²FedRec is the first framework tailored for both data-level and edge-level privacy problems in relationship graph-enhanced federated recommendation.
- We develop a user-server collaborative mechanism for relationship graph generation and user-specific embedding aggregation while offering multi-level privacy guarantees. In particular, we design an embedding-shared local graph construction module and a noisy global graph-guided aggregation module to safeguard data-level and edge-level privacy against semi-honest adversaries, respectively.
- By providing rigorous theoretical analysis, we prove that P²FedRec can guarantee data-level and edge-level privacy in federated recommendation scenarios. Furthermore, we evaluate P²FedRec on five benchmark recommendation datasets, and the experimental results demonstrate its superior performance compared to state-of-the-art works.

2 Preliminaries

2.1 Differential Privacy

Differential Privacy (DP) is a mathematical method that can quantify and minimize the risk of individual data leakage. By constraining the influence of each individual record in a database it ensures that the existence of any record cannot be probabilistically inferred from the released data. Typically, this goal is achieved by adding noise to the data or the algorithm. The most widely used form of DP is ϵ -differential privacy, which is defined as follows.

DEFINITION 2.1. (ϵ -DP). *Let $\mathcal{D}$ and $\mathcal{S}$ denote the spaces of all possible databases and outputs, respectively. A randomized algorithm $\mathcal{R} : \mathcal{D} \rightarrow \mathcal{S}$ is said to be ϵ -differential privacy if, for any pair of databases $D, D' \in \mathcal{D}$ that differ by only one record, and for any output $S \in \mathcal{S}$, the following condition is satisfied:*

$$\frac{\Pr[\mathcal{R}(D) = S]}{\Pr[\mathcal{R}(D') = S]} \leq \exp(\epsilon). \quad (1)$$

If each data owner applies a randomized algorithm (satisfying Definition 2.1) to their data before uploading to the server, ϵ -local DP (LDP) can be achieved. We then present a key property of LDP, i.e., post-processing immunity. This property guarantees that any subsequent processing performed on the perturbed data cannot diminish the original privacy protection guarantees.

THEOREM 2.1. (Post-Processing) *Assume $\mathcal{R} : \mathcal{D} \rightarrow \mathcal{S}$ is an algorithm satisfies ϵ -LDP, and $\mathcal{F} : \mathcal{S} \rightarrow \mathcal{S}'$ denote an arbitrary mapping, then $\mathcal{F} \circ \mathcal{R} : \mathcal{D} \rightarrow \mathcal{S}'$ still satisfies ϵ -LDP, where $\circ$ is a composition of $\mathcal{F}$ and $\mathcal{R}$.*

In this paper, LDP is primarily used to protect relationship graphs (denoted as adjacency lists) from the threat of servers. To this end, we provide the definition of ϵ -edge LDP as follows.

DEFINITION 2.2. (*ϵ -edge LDP*) A randomized algorithm $\mathcal{A} : \{0, 1\}^n \rightarrow \mathcal{S}$ is said to achieve ϵ -edge differential privacy if, given any pair of neighboring lists $l, l' \in \{0, 1\}^n$ differing in exactly one entry (i.e., $\|l - l'\|_{1,1} = 1$), and for any output $S \in \mathcal{S}$, the following condition is satisfied:

$$\frac{\Pr[\mathcal{A}(l) = S]}{\Pr[\mathcal{A}(l') = S]} \leq \exp(\epsilon). \quad (2)$$

2.2 Additive Secret Sharing

Given two messages $x, y \in \mathbb{Z}_{2^k}$, a 2-out-of-2 additive secret sharing (ASS) samples two uniform share $\langle x \rangle_1, \langle y \rangle_1 \in \mathbb{Z}_{2^k}$ and sets $\langle x \rangle_2 = x - \langle x \rangle_1 \bmod 2^k$, $\langle y \rangle_2 = y - \langle y \rangle_1 \bmod 2^k$. The shares $\langle x \rangle_1, \langle y \rangle_1$ and $\langle x \rangle_2, \langle y \rangle_2$ will be distributed to two parties $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively. Unless otherwise specified, all subsequent ASS-based computations are performed in $\mathbb{Z}_{2^k}$ and the modulo operation is omitted for brevity. Next we introduce the basic operations on these shares.

- **Addition/Subtraction.** $\mathcal{P}_1$ and $\mathcal{P}_2$ can execute addition or subtraction operation locally, i.e., $\langle z \rangle_i = \langle x \rangle_i \pm \langle y \rangle_i$, $i \in \{1, 2\}$.
- **Multiplication.** The multiplication requires the help of Beaver triples. A Beaver triple is a multiplication triple $\{r^{(1)}, r^{(2)}, s\}$, secret-shared between $\mathcal{P}_1$ and $\mathcal{P}_2$, where $s = r^{(1)} \cdot r^{(2)}$. Each party $\mathcal{P}_i$ begins by locally computing $\langle e \rangle_i = \langle x \rangle_i - \langle r^{(1)} \rangle_i$, $\langle f \rangle_i = \langle y \rangle_i - \langle r^{(2)} \rangle_i$ and then exchange $\langle e \rangle_i, \langle f \rangle_i$ to reconstruct e and f together. Finally, each party computes the share of the multiplication result independently: $\mathcal{P}_1$ calculates $\langle z \rangle_1 = f \cdot \langle r^{(1)} \rangle_1 + e \cdot \langle r^{(2)} \rangle_1 + \langle s \rangle_1$, while $\mathcal{P}_2$ computes $\langle z \rangle_2 = e \cdot f + f \cdot \langle r^{(1)} \rangle_2 + e \cdot \langle r^{(2)} \rangle_2 + \langle s \rangle_2$.

2.3 Homomorphic Encryption

Homomorphic encryption (HE) [1] is a specialized cryptographic technique whose core property is the ability to perform certain computations directly on ciphertext, such that, once decrypted, the result is identical to the outcome of applying the same operations to the original plaintext. In this paper, we use an additive homomorphic encryption, denoted as $\mathcal{E}$, and its key properties are shown below:

$$\mathcal{E}(x_1) \cdot \mathcal{E}(x_2) = \mathcal{E}(x_1 + x_2), \quad \mathcal{E}(x_1)^a = \mathcal{E}(a \cdot x_1), \quad (3)$$

where $x_1, x_2 \in \mathbb{Z}_N$ represents two messages, and $a \in \mathbb{Z}_N$ represents a constant.

3 Problem Formulation and Threat Model

3.1 Problem Formulation

Let $\mathcal{U}$ and $\mathcal{M}$ denote the sets of users and items in a typical FedRec scenario, respectively. The interaction data of each user u_i is represented as $\langle i, m, y_{im} \rangle$, where $y_{im} \in \{0, 1\}$, $i \in [1, |\mathcal{U}|]$, and $m \in [1, |\mathcal{M}|]$. $y_{im} = 1$ indicates that an interaction exists between user u_i and the m -th item, while $y_{im} = 0$ means no interaction occurs. Following [47], we use a neural recommendation model θ that consists of three components: a scoring function f , a user embedding $\mathbf{p}$, and an item embedding $\mathbf{g}$. The scoring function takes the user and item embeddings as input to generate recommendation scores, and the item embedding acts as a shared component that transfers common knowledge across users.

To accommodate the diversity in user preferences in FedRec scenarios, we aim to enable each user to train a personalized local model tailored to their individual characteristics. To further enhance the ability of these models to capture user-specific preferences, we incorporate relationship graphs into the FedRec system. Specifically, we let two servers compute similarity scores between all users' item embeddings and construct a user relationship graph $\mathcal{L} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{U}|}$, where $\mathcal{L}_{ij} = 1$ indicates that users u_i and u_j share similar preferences, and $\mathcal{L}_{ij} = 0$ otherwise. Then, for each user,

two servers aggregate the information of their neighbors to generate user-specific item embeddings $\hat{g}_i$, $i \in [1, |\mathcal{U}|]$. The optimal model parameters are learned by optimizing the objective function:

$$\min_{\{\theta_1, \dots, \theta_{|\mathcal{U}|}\}} \sum_{i=1}^{|\mathcal{U}|} \mathcal{L}_i(\theta_i) + \beta \mathcal{R}(\mathbf{g}_i, \hat{\mathbf{g}}_i), \quad (4)$$

where $\theta_i = \{\mathbf{p}_i, \mathbf{g}_i, \mathbf{f}_i\}$ represents the model parameters for user u_i , $\mathcal{L}_i(\theta_i)$ is the corresponding loss function, $\mathcal{R}(\cdot, \cdot)$ is a regularization term, and β is a weighting coefficient.

We aim to design a FedRec system that leverages user relationship graphs to enhance personalized modeling while achieving the following goals:

- **Guarantee data-level privacy.** Throughout the model training process, the two servers are unable to reconstruct the item embeddings of users. Concurrently, users are restricted to accessing only their own information and cannot obtain item embeddings about other users.
- **Guarantee edge-level privacy.** The two servers cannot determine which users are linked in the relationship graph. Meanwhile, the users cannot know which users are linked to them either; in other words, they do not know the identities of their neighbors.

3.2 Threat Model

P²FedRec introduces a two-server model, where $\mathcal{P}_1$ and $\mathcal{P}_2$ are deployed in different trust domains. Compared with a single-server model, it has the following advantages. In a single-server model, to protect data-level privacy, it is necessary to protect item embeddings using homomorphic encryption or differential privacy. However, this often leads to excessive overhead or compromised model accuracy. In the two-server model, secret sharing can be used to protect the privacy of item embeddings, effectively avoiding the aforementioned issues. Therefore, this model has garnered widespread attention recently [10, 14].

We assume all entities, including two servers $\mathcal{P}_1$ and $\mathcal{P}_2$ and all user are honest but curious, which is a common assumption in prior work [15, 20, 21]. This implies that they will follow the protocol but may try to infer the interaction data or relationship of (other) users from their accessible information. Besides, consistent with prior two-server-based work [10, 14], we assume non-collusion between the two servers. Notably, in addition to the academic community, this multi-server secure computation model has also gained widespread acceptance in industrial communities. For instance, with a non-colluding multi-server architecture, Mozilla develops the Firefox Origin Telemetry system, which supports secure telemetry data collection¹; Apple and Google co-develop an intelligent early-warning system to generate potential COVID-19 exposure alerts for users via a multi-server architecture².

4 Overview

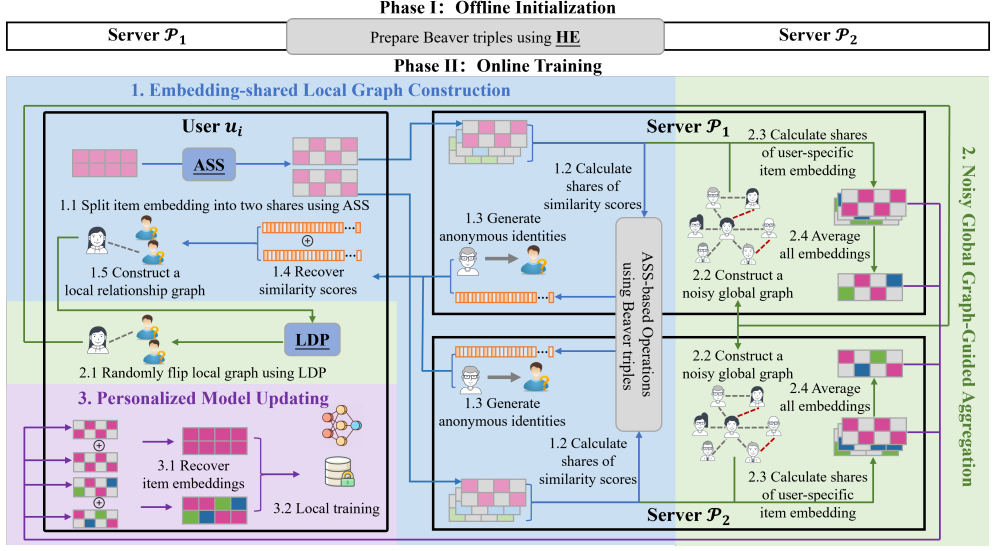
P²FedRec enables personalized modeling without compromising data-level and edge-level privacy in recommendation systems. As shown in Fig. 2, P²FedRec has two phases. The phase I is executed only once, primarily dedicated to preparing Beaver triples for the subsequent training phase. The phase II includes the entire process from constructing user relationship graphs to training personalized models. In this section, we describe the major building components and workflow of P²FedRec.

4.1 Offline Initialization Phase

In this phase, two servers $\mathcal{P}_1$ and $\mathcal{P}_2$ are required to prepare Beaver triples to support ASS-based multiplication operations in the subsequent online training phase. We begin by having parties $\mathcal{P}_1$ and $\mathcal{P}_2$ randomly select secret shares $\langle r^{(1)} \rangle_1, \langle r^{(2)} \rangle_1, \dots, \langle r^{(|\mathcal{U}|)} \rangle_1$ and $\langle r^{(1)} \rangle_2, \langle r^{(2)} \rangle_2, \dots, \langle r^{(|\mathcal{U}|)} \rangle_2$

¹<https://blog.mozilla.org/security/2019/06/06/next-steps-in-privacy-preserving-telemetry-with-prio/>

²<https://covid19.apple.com/contacttracing>

Fig. 2. An Overview of P²FedRec.**Algorithm 1:** Prepare Beaver triples with HE**Input:** $\langle r^{(1)} \rangle_1, \dots, \langle r^{(|\mathcal{U}|)} \rangle_1$ and $\langle r^{(1)} \rangle_2, \dots, \langle r^{(|\mathcal{U}|)} \rangle_2$ **Output:** $\langle s^{(i,j)} \rangle_1, \langle s^{(i,j)} \rangle_2, i, j \in \{1, \dots, |\mathcal{U}|\}$ with $i > j$

```

1  $\mathcal{P}_1$ : Encrypt  $\langle r^{(1)} \rangle_1, \dots, \langle r^{(|\mathcal{U}|)} \rangle_1$  into  $\mathcal{E}(\langle r^{(1)} \rangle_1), \dots, \mathcal{E}(\langle r^{(|\mathcal{U}|)} \rangle_1)$  and sends ciphertexts to  $\mathcal{P}_2$ ;
2 for  $i \leftarrow 1, \dots, |\mathcal{U}|$  do
3   for  $j \leftarrow 1, \dots, i-1$  do
4      $\mathcal{P}_2$ : Generate a random value  $c^{(i,j)} \leftarrow \{0, 1\}^{2 \cdot k + 1 + \sigma}$ , where  $\sigma$  denotes a statistical
       security parameter;
5      $\mathcal{P}_2$ : Calculate  $\langle s^{(i,j)} \rangle_2 = \langle r^{(i)} \rangle_2 \cdot \langle r^{(j)} \rangle_2 - c^{(i,j)}$ ;
6      $\mathcal{P}_2$ : Calculate  $\mathcal{E}(v^{(i,j)}) = \mathcal{E}(\langle r^{(i)} \rangle_1 \cdot \langle r^{(j)} \rangle_2 + \langle r^{(j)} \rangle_1 \cdot \langle r^{(i)} \rangle_2 + c^{(i,j)})$ ;
7   end
8 end
9  $\mathcal{P}_2$ : Send  $\mathcal{E}(v^{(i,j)})$ ,  $i, j \in \{1, \dots, |\mathcal{U}|\}$  with  $i > j$  to  $\mathcal{P}_1$ ;
10 for  $i \leftarrow 1, \dots, |\mathcal{U}|$  do
11   for  $j \leftarrow 1, \dots, i-1$  do
12      $\mathcal{P}_1$ : Decrypt  $\mathcal{E}(v^{(i,j)})$  to obtain  $v^{(i,j)}$ ;
13      $\mathcal{P}_1$ : Calculate  $\langle s^{(i,j)} \rangle_1 = \langle r^{(i)} \rangle_1 \cdot \langle r^{(j)} \rangle_1 + v^{(i,j)} \bmod 2^k$ ;
14   end
15 end

```

respectively, where each $r^{(i)}$ satisfies $r^{(i)} = \langle r^{(i)} \rangle_1 + \langle r^{(i)} \rangle_2$ for $i \in 1, \dots, |\mathcal{U}|$. Following the procedure in Algorithm 1, $\mathcal{P}_1$ and $\mathcal{P}_2$ then compute shares $\langle s^{(i,j)} \rangle_1$ and $\langle s^{(i,j)} \rangle_2$ for all unique pairs (i, j) where $1 \leq j < i \leq |\mathcal{U}|$, ensuring that $s^{(i,j)} = \langle s^{(i,j)} \rangle_1 + \langle s^{(i,j)} \rangle_2 = r^{(i)} \cdot r^{(j)}$. This process yields $|\mathcal{U}|(|\mathcal{U}| - 1)/2$ triples $(r^{(i)}, r^{(j)}, s^{(i,j)})$ securely shared between $\mathcal{P}_1$ and $\mathcal{P}_2$, corresponding to all distinct pairs of indices with $i > j$.

4.2 Online Training Phase

This phase consists of three modules: Firstly, in the *embedding-shared local graph construction* module, each user locally constructs an ego relationship graph of other users with similar preferences. Secondly, in the *noisy global graph-guided aggregation* module, each user uploads a DP-perturbed local graph to the servers, which construct a global graph and perform user-specific item embedding aggregation. Finally, in the *personalized model updating* module, each user trains their local models using the user-specific item embeddings.

4.2.1 Embedding-shared Local Graph Construction. In this module, two servers assist users in identifying neighbors with similar preferences and building local relationship graphs. Specifically, each user first splits their item embeddings into two shares using ASS and distributes these shares to servers $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively (**step 1.1**). $\mathcal{P}_1$ and $\mathcal{P}_2$ then leverage pre-generated Beaver triplets to perform ASS-based operations, thereby obtaining shares of the similarity scores between any pair of item embeddings (**step 1.2**). Notably, to guarantee the edge-level privacy on the user side, we let $\mathcal{P}_1$ and $\mathcal{P}_2$ assign each user an anonymous identity to decouple users from their corresponding item embeddings (**step 1.3**). Subsequently, $\mathcal{P}_1$ and $\mathcal{P}_2$ will respectively send to each user the similarity shares between this user's item embedding and that of others, and the corresponding anonymized identities of others. Finally, each user recovers all similarity scores (**step 1.4**), and constructs a local relationship graph that consists of anonymized neighbors with similar preferences (**step 1.5**).

4.2.2 Noisy Global Graph-Guided Aggregation. In this module, the servers aggregate user-specific item embeddings based on the relationship graphs. Specifically, to protect edge-level privacy on the server side, each user uses LDP to perturb the local graph and only uploads this noisy version (**step 2.1**). To enhance the usability of these perturbed graphs, $\mathcal{P}_1$ and $\mathcal{P}_2$ then independently use Bayesian estimation to construct a denoised global version (**step 2.2**). Subsequently, for each user, they aggregate item embedding shares of their neighbors to generate user-specific embedding shares (**step 2.3**). Moreover, to capture global popularity trends, $\mathcal{P}_1$ and $\mathcal{P}_2$ average all user-specific embedding shares, respectively (**step 2.4**).

4.2.3 Personalized Model Updating. After receiving their respective shares from servers, each user first recovers the user-specific embedding and the global embedding (**step 3.1**), and then uses these embeddings to conduct personalized local training (**step 3.2**).

5 Building Components

In this section, we present a detailed description of three building components of the online training phase, i.e., *embedding-shared local graph construction*, *noisy global graph-guided aggregation*, and *personalized model updating*, and outline their implementation in Algorithm 2.

5.1 Embedding-shared Local Graph Construction

In this part, we will introduce how to enable the server to assist users in constructing their local relationship graphs.

5.1.1 User-side Embedding Splitting. At iteration t , the item embedding of user u_i is represented by an H -dimensional vector $\mathbf{g}_i^t \in \mathbb{R}^H$, where $\mathbf{g}_i^t[h]$ denotes its h -th element. To enable secure similarity computation, u_i first preprocesses the embedding by locally normalizing $\mathbf{g}_i^t$ to obtain $\bar{\mathbf{g}}_i^t$, and then splits both vectors into secret shares for servers $\mathcal{P}_1$ and $\mathcal{P}_2$ using ASS. A key technical challenge arises from the fact that real-world embeddings typically contain floating-point values while secret sharing operates over integers. To bridge this gap, we follow [29] by employing Q -bit fixed-point encoding, representing each element $\mathbf{g}_i^t[h]$ as $\lfloor \mathbf{g}_i^t[h] \cdot 2^Q \rfloor$. This encoding requires special handling

Algorithm 2: P²FedRec (Online Training Phase)

Input: Initial local models $\{(\mathbf{p}_i, \mathbf{g}_i, \mathbf{f}_i)\}_{i=1}^{|\mathcal{U}|}$, rounds T , learning rate η , hyperparameter α , auxiliary triples $(r^{(i)}, r^{(j)}, s^{(i,j)})$, $i, j \in [1, |\mathcal{U}|]$ with $i > j$

1 . Output: Users' personalized local models

2 for each round $t \leftarrow 1, 2, \dots, T$ **do**

3 *// embedding-shared local graph construction;*

4 **for each user** $i \leftarrow 1, 2, \dots, |\mathcal{U}|$ **in parallel do**

5 | Split item embedding into $\langle \mathbf{g}_i^t \rangle_1$ and $\langle \mathbf{g}_i^t \rangle_2$;

6 **end**

7 $\mathcal{P}_1$ and $\mathcal{P}_1$ collaboratively calculate the similarity shares of item embeddings with Eq. (5) and (6);

8 **for each user** $i \leftarrow 1, 2, \dots, |\mathcal{U}|$ **in parallel do**

9 | Construct a local relationship graph consisting of anonymized neighbors with Eq. (8);

10 **end**

11 *// Noisy global graph-guided aggregation;*

12 **for each user** $i \leftarrow 1, 2, \dots, |\mathcal{U}|$ **in parallel do**

13 | Perturbed local graph with Eq. (9) and (10);

14 **end**

15 $\mathcal{P}_1$ and $\mathcal{P}_2$ reconstruct a global relationship graph respectively with Eq. (17);

16 $\mathcal{P}_1$ and $\mathcal{P}_2$ calculate shares of user-specific and global item embeddings, respectively, with Eq. (18) and (19);

17 *// Personalized model updating;*

18 **for each user** $i \leftarrow 1, 2, \dots, |\mathcal{U}|$ **in parallel do**

19 | Recover user-specific and global item embeddings with Eq. (20);

20 | Update personalized local model with Eq. (22);

21 **end**

22 end

during multiplication operations. When two encoded values (each scaled by 2^Q) are multiplied, the result becomes scaled by 2^{2Q} , necessitating rescaling through the truncation technique described in [29]. For notational simplicity, we omit the encoding process in the following description and assume all subsequent mentions of $\mathbf{g}_i^t$ and related vectors refer to their fixed-point encoded counterparts in $\mathbb{Z}_{2^k}$. Then, the sharing process proceeds as follows: u_i randomly generates vectors $\langle \mathbf{g}_i^t \rangle_1, \langle \bar{\mathbf{g}}_i^t \rangle_1 \in \mathbb{Z}_{2^k}^H$, computes their complementary shares as $\langle \mathbf{g}_i^t \rangle_2 = \mathbf{g}_i^t - \langle \mathbf{g}_i^t \rangle_1$ and $\langle \bar{\mathbf{g}}_i^t \rangle_2 = \bar{\mathbf{g}}_i^t - \langle \bar{\mathbf{g}}_i^t \rangle_1$, and distributes the share pairs $(\langle \mathbf{g}_i^t \rangle_1, \langle \bar{\mathbf{g}}_i^t \rangle_1)$ to $\mathcal{P}_1$ and $(\langle \mathbf{g}_i^t \rangle_2, \langle \bar{\mathbf{g}}_i^t \rangle_2)$ to $\mathcal{P}_2$, respectively.

5.1.2 Server-side Similarity Calculation. Upon receiving the share pairs $(\langle \mathbf{g}_i^t \rangle_1, \langle \bar{\mathbf{g}}_i^t \rangle_1)$ and $(\langle \mathbf{g}_i^t \rangle_2, \langle \bar{\mathbf{g}}_i^t \rangle_2)$ from all users u_i ($i \in [1, |\mathcal{U}|]$), $\mathcal{P}_1$ and $\mathcal{P}_2$ utilize pre-generated Beaver triples to securely compute similarity shares between the item embeddings of any two users. There are many commonly used similarity metrics, and here we adopt *cosine similarity*. Specifically, for each user $i \in [1, |\mathcal{U}|]$, $\mathcal{P}_1$ generates a length- H vector $\langle \mathbf{r}^{(i)} \rangle_1$ with each element set to $\langle \mathbf{r}^{(i)} \rangle_1$ and computes $\langle \mathbf{e}_i^t \rangle_1 = \langle \bar{\mathbf{g}}_i^t \rangle_1 - \langle \mathbf{r}^{(i)} \rangle_1$. Similarly, $\mathcal{P}_2$ constructs $\langle \mathbf{r}^{(i)} \rangle_2$ and derives $\langle \mathbf{e}_i^t \rangle_2$ in an analogous manner. The two parties then exchange their respective shares $\{\langle \mathbf{e}_i^t \rangle_1\}_{i=1}^{|\mathcal{U}|}$ and $\{\langle \mathbf{e}_i^t \rangle_2\}_{i=1}^{|\mathcal{U}|}$, allowing them to reconstruct the full embeddings $\mathbf{e}_i^t = \langle \mathbf{e}_i^t \rangle_1 + \langle \mathbf{e}_i^t \rangle_2$ for each user. Using these reconstructed values, $\mathcal{P}_1$ proceeds to compute the secret-shared similarity score between any pair of users u_i and u_j

($i, j \in [1, |\mathcal{U}|]$) as:

$$\begin{aligned} \langle z_{i,j}^t \rangle_1 = & \sum_{h=1}^H (\mathbf{e}_i^t[h] \cdot \mathbf{e}_j^t[h] + \mathbf{e}_i^t[h] \cdot \langle \mathbf{g}_j^t[h] \rangle_1 \\ & + \mathbf{e}_j^t[h] \cdot \langle \mathbf{g}_i^t[h] \rangle_1 + \langle s_{(i,j)} \rangle_1). \end{aligned} \quad (5)$$

Concurrently, $\mathcal{P}_2$ performs

$$\langle z_{i,j}^t \rangle_2 = \sum_{h=1}^H (\mathbf{e}_i^t[h] \cdot \langle \mathbf{g}_j^t[h] \rangle_2 + \mathbf{e}_j^t[h] \cdot \langle \mathbf{g}_i^t[h] \rangle_2 + \langle s_{(i,j)} \rangle_2). \quad (6)$$

Notably, the user relationship graph here is undirected, i.e., $z_{i,j}^t = z_{j,i}^t$. Therefore, we exclusively consider $i > j$ in our computation.

Allowing users to independently determine their neighbor number during local graph construction would complicate the formation of a consistent global graph on the server side. This inconsistency arises because a connection (e.g., between users u_i and u_j) might be established by u_i but not by u_j , leading to mismatches. To address this, we introduce a unified yet flexible neighbor selection strategy. Specifically, $\mathcal{P}_1$ and $\mathcal{P}_2$ compute the average of all secret-shared similarity scores:

$$\begin{aligned} \langle \bar{z}^t \rangle_1 = & \frac{2}{|\mathcal{U}|(|\mathcal{U}|-1)} \sum_{i=1}^{|\mathcal{U}|} \sum_{j=1}^{i-1} \langle z_{i,j}^t \rangle_1, \\ \langle \bar{z}^t \rangle_2 = & \frac{2}{|\mathcal{U}|(|\mathcal{U}|-1)} \sum_{i=1}^{|\mathcal{U}|} \sum_{j=1}^{i-1} \langle z_{i,j}^t \rangle_2. \end{aligned} \quad (7)$$

and subsequently apply a hyperparameter α to fine-tune the neighbor selection process.

At this point, the server should provide each user with their secret-shared similarity scores and associated identifiers to facilitate local neighbor identification. However, if the identifiers of users are directly shared, each user could infer others' preferences from the similarity scores, leading to edge-level privacy leakage on the user side. To address this, we use a dynamic pseudo-anonymization mechanism where servers generate a fresh set of anonymized identifiers for all users in each round. For example, $\mathcal{P}_1$ and $\mathcal{P}_2$ can negotiate to implement this through directly using predefined sequential identifiers (e.g., user 1, user 2, ..., user $|\mathcal{U}|$ as used throughout this paper) or using hash-based user mapping to establish these temporary identities. The core security rationale behind this round-based identifier renewal lies in enforcing unlinkability at the user side, ensuring that the same user cannot be correlated across different rounds. With this dynamic pseudonymization mechanism in place, $\mathcal{P}_1$ and $\mathcal{P}_2$ will finally deliver to each user u_i three pieces of information: the secret-shared similarity scores between u_i and all other users, the average of all secret-shared scores, and the corresponding anonymized identifiers generated for the current iteration. This ensures users can perform neighbor identification while protecting edge-level privacy.

5.1.3 User-side Local graph Construction. Upon receiving messages described above, each user $u_i, i \in [1, |\mathcal{U}|]$ first recovers $z_{i,j}^t = \langle z_{i,j}^t \rangle_1 + \langle z_{i,j}^t \rangle_2, j \in [1, |\mathcal{U}|] \wedge j \neq i$, $\alpha \bar{z}^t = \alpha(\langle \bar{z}^t \rangle_2 + \langle \bar{z}^t \rangle_2)$, and then constructs a local graph (denoted as an adjacency list) as below

$$\mathcal{L}_{ij} = \begin{cases} 1, & z_{i,j}^t \geq \alpha \bar{z}^t \\ 0, & \text{otherwise} \end{cases}, \quad (8)$$

where, $\mathcal{L}_{ii} = 0$. Here, each user u_i only knows the temporary identifiers of other users but cannot know their real identities. This ensures that each user u_i cannot link the anonymous neighbors in their locally constructed graph to actual individuals, thereby preserving relationship (i.e., edge-level) privacy on the user side.

5.2 Noisy Global Graph-Guided Aggregation

In this part, we detail how the servers perform effective embedding aggregation using the noise-injected local graphs uploaded by users.

5.2.1 User-side Noise Injection. Users should upload local graphs to the server for embedding aggregation. However, since the servers know the mapping between anonymous identifiers and real users, directly uploading the raw local graphs would allow the server to infer which real-world users have similar preferences, leading to edge-level privacy leakage on the server side. To solve this problem, we let users use local differential privacy to perturb the adjacency list and split the privacy budget ϵ for degree information d_i and adjacency list $\mathcal{L}_i$, where $\epsilon_d = \gamma\epsilon$, $\gamma \in [0, 1]$ and $\epsilon_a = (1 - \gamma)\epsilon$ are spent on degree d_i and adjacency list $\mathcal{L}_i$, respectively. Such approaches of allocating the privacy budget to degree information and adjacency lists have been effectively validated and used in previous studies [44, 51]. Notably, like most LDP-based federated learning works [38, 50], all users in our work use the same ϵ . For d_i , we add noise to it using the widely-used Laplace mechanism $l_i \sim \text{Laplace}(0, 1/\epsilon_d)$ as below

$$\tilde{d}_i \leftarrow d_i + l_i. \quad (9)$$

And for each element in $\mathcal{L}_i$, we perform random flipping [37] on it with probability $1/(1 + \exp(\epsilon_a))$, i.e.,

$$\tilde{\mathcal{L}}_{ij} = \begin{cases} \mathcal{L}_{ij}, & \text{with probability } \frac{\exp(\epsilon_a)}{1 + \exp(\epsilon_a)} \\ 1 - \mathcal{L}_{ij}, & \text{with probability } \frac{1}{1 + \exp(\epsilon_a)} \end{cases}. \quad (10)$$

Finally, the perturbed degree $\tilde{d}_i$ and adjacency list $\tilde{\mathcal{L}}_i$ will be uploaded to $\mathcal{P}_1$ and $\mathcal{P}_2$, respectively.

5.2.2 Server-side Denoising. $\mathcal{P}_1$ and $\mathcal{P}_2$ first respectively assemble $\tilde{d}_1, \tilde{d}_2, \dots, \tilde{d}_{|\mathcal{U}|}$ and $\tilde{\mathcal{L}}_1, \tilde{\mathcal{L}}_2, \dots, \tilde{\mathcal{L}}_{|\mathcal{U}|}$ into a degree sequence $\tilde{d} \in \mathbb{R}^{|\mathcal{U}|}$ and an adjacency matrix $\tilde{\mathcal{L}} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{U}|}$, respectively. To reduce the impact of noise, $\mathcal{P}_1$ and $\mathcal{P}_2$ can use the received noisy degree sequence as the prior and the noisy adjacency matrix as the evidence to calculate the posterior probabilities of each potential link [51]. Firstly, given $\tilde{d} \in \mathbb{R}^{|\mathcal{U}|}$, $\mathcal{P}_1$ and $\mathcal{P}_2$ respectively use the β -model [7, 18] to estimate the probability of the link. For a vector $\beta = (\beta_1, \beta_2, \dots, \beta_{|\mathcal{U}|}) \in \mathbb{R}^{|\mathcal{U}|}$, this model assumes that in a random undirected simple graph with $|\mathcal{U}|$ vertices, the probability of forming an edge between any pair of vertices u_i and u_j (where $1 \leq j < i \leq |\mathcal{U}|$) is $p_{ij} = \frac{\exp(\beta_i + \beta_j)}{1 + \exp(\beta_i + \beta_j)}$, and this probability is independent of the formation of all other edges. The likelihood of encountering the actual degree sequence $d = (d_1, \dots, d_{|\mathcal{U}|})$ is captured by

$$L_d(\beta) = \frac{\exp(\sum_j \beta_j d_j)}{\prod_{j < i} (1 + \exp(\beta_i + \beta_j))}. \quad (11)$$

Maximizing the likelihood function $L_d(\beta)$ can infer the value of β . However, $\mathcal{P}_1$ and $\mathcal{P}_2$ can only obtain the noisy degree sequence $\tilde{d}$, not the true degree sequence d , so it can only maximize the likelihood function of $L_{\tilde{d}}(\beta)$ using observed $\tilde{d}$ as follows.

$$\tilde{d}_j = \sum_{i \neq j} \frac{\exp(\hat{\beta}_i + \hat{\beta}_j)}{1 + \exp(\hat{\beta}_i + \hat{\beta}_j)}, \quad j = 1, 2, \dots, |\mathcal{U}|, \quad (12)$$

Then, $\mathcal{P}_1$ and $\mathcal{P}_2$ can respectively calculate the solution of the above maximum likelihood estimate (MLE) using the function $\phi_d : \mathbb{R}^{|\mathcal{U}|} \rightarrow \mathbb{R}^{|\mathcal{U}|}$ [7], where

$$\phi_{\tilde{d}}(b)_j = \log(\tilde{d}_j) - \log \left(\sum_{i \neq j} \frac{1}{\exp(-b_i) + \exp(b_j)} \right), \quad (13)$$

and Chatterjee et al. [7] have already proved that the MLE solution is a fixed point of ϕ . For every pair of users u_i, u_j , $\mathcal{P}_1$ and $\mathcal{P}_2$ respectively have two bits, $\tilde{\mathcal{L}}_{ij}$ and $\tilde{\mathcal{L}}_{ji}$, which indicate the likelihood of a connection between them. Using the flip probability $pro = \frac{1}{1+\exp(\epsilon_a)}$, $\mathcal{P}_1$ and $\mathcal{P}_2$ can respectively compute the probability of observing the received bits $(\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji})$ under the condition that u_i and u_j are linked in the true graph as below

$$q_{ij} = \begin{cases} pro^2, & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (0, 0) \\ pro(1-pro), & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) / (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (0, 1) \\ (1-pro)^2, & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (1, 1) \end{cases} \quad (14)$$

Similarly, if there is no link between u_i and u_j in the true graph, the probability of observing $(\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji})$ is

$$q'_{ij} = \begin{cases} (1-pro)^2, & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (0, 0) \\ pro(1-pro), & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) / (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (0, 1) \\ pro^2, & (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji}) = (1, 1) \end{cases} \quad (15)$$

Therefore, based on Bayes rule, the posterior probability of the link can be obtained as follows

$$\bar{p}_{ij} = \Pr[(\mathcal{L}_{ij}, \mathcal{L}_{ji}) = (1, 1) | (\tilde{\mathcal{L}}_{ij}, \tilde{\mathcal{L}}_{ji})] = \frac{q_{ij}p_{ij}}{q_{ij}p_{ij} + q'_{ij}(1-p_{ij})}. \quad (16)$$

Then, $\mathcal{P}_1$ and $\mathcal{P}_2$ can respectively construct the global graph with adjacency matrix $\bar{\mathcal{L}} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{U}|}$ as below

$$\bar{\mathcal{L}}_{ij} = \bar{\mathcal{L}}_{ji} = \begin{cases} 1, & \bar{p}_{ij} > 0.5 \\ 0, & \text{otherwise} \end{cases} \quad (17)$$

5.2.3 User-specific Embedding Aggregation. In this step, with the constructed global graph, $\mathcal{P}_1$ and $\mathcal{P}_2$ aggregate item embedding shares from neighbors with similar preferences for each user $u_i, i \in [1, |\mathcal{U}|]$ via a simple yet effective mechanism, obtaining shares of user-specific item embeddings as follows:

$$\langle \hat{\mathbf{g}}_i^t \rangle_1 = \frac{1}{|\mathcal{B}_i|} \sum_{j \in \mathcal{B}_i} \langle \mathbf{g}_j^t \rangle_1, \quad \langle \hat{\mathbf{g}}_i^t \rangle_2 = \frac{1}{|\mathcal{B}_i|} \sum_{j \in \mathcal{B}_i} \langle \mathbf{g}_j^t \rangle_2. \quad (18)$$

where $\mathcal{B}_i$ denote the set of neighbors of user u_i in the global graph. Then, to represent the global popular preference, $\mathcal{P}_1$ and $\mathcal{P}_2$ respectively perform an average over all shares of user-specific item embeddings to obtain the shares of global item embedding as

$$\langle \hat{\mathbf{g}}_{average}^t \rangle_1 = \frac{1}{|\mathcal{U}|} \sum_{i=1}^{|\mathcal{U}|} \langle \hat{\mathbf{g}}_i^t \rangle_1, \quad \langle \hat{\mathbf{g}}_{average}^t \rangle_2 = \frac{1}{|\mathcal{U}|} \sum_{i=1}^{|\mathcal{U}|} \langle \hat{\mathbf{g}}_i^t \rangle_2. \quad (19)$$

Finally, the shares of user-specific item embeddings will be sent to the corresponding users, and the shares of global item embedding will be distributed to all users.

5.3 Personalized Model Updating

For each user u_i , upon receiving the embedding shares $\langle \hat{\mathbf{g}}_{average}^t \rangle_1, \langle \hat{\mathbf{g}}_{average}^t \rangle_2, \langle \hat{\mathbf{g}}_i^t \rangle_1$, and $\langle \hat{\mathbf{g}}_i^t \rangle_2$, they first reconstruct the global and user-specific item embeddings by computing

$$\hat{\mathbf{g}}_{average}^t = \langle \hat{\mathbf{g}}_{average}^t \rangle_1 + \langle \hat{\mathbf{g}}_{average}^t \rangle_2, \quad \hat{\mathbf{g}}_i^t = \langle \hat{\mathbf{g}}_i^t \rangle_1 + \langle \hat{\mathbf{g}}_i^t \rangle_2. \quad (20)$$

Subsequently, the local item embeddings are updated to the current global average embeddings ($\mathbf{g}_i^t \leftarrow \hat{\mathbf{g}}_{average}^t$), while the user embedding $\mathbf{p}_i^t$ and the score function $\mathbf{f}_i^t$ remain unchanged from the

previous iteration. To ensure personalization, each user u_i further regularizes $\mathbf{g}_i^t$ to align it with their personalized embedding $\hat{\mathbf{g}}_i^t$. This is formalized through the following objective:

$$\begin{aligned} \mathcal{L}_i &= \mathcal{L}(\theta_i; y_{im}, \tilde{y}_{im}) + \beta \mathcal{R}(\mathbf{g}_i^t, \hat{\mathbf{g}}_i^t), \\ \mathcal{L}(\theta_i; y_{im}, \tilde{y}_{im}) &= - \sum_{m=1}^{|\mathcal{M}_i|} \log \tilde{y}_{im} - \sum_{m=1}^{|\mathcal{M}_i^-|} \log(1 - \tilde{y}_{im}), \end{aligned} \quad (21)$$

where $\tilde{y}_{im}$ denotes a prediction, $\mathcal{M}_i$ and $\mathcal{M}_i^-$ denote the sets of positive and sampled negative user-item interactions, respectively, and $\mathcal{R}(\cdot, \cdot)$ is implemented as mean square error. The negative samples in $\mathcal{M}_i^-$ are efficiently constructed by randomly selecting items from the unobserved interaction set at a predefined sampling ratio. Finally, each user u_i optimizes their local model via stochastic gradient descent:

$$\theta_i^{t+1} = \theta_i^t - \eta \frac{\partial \mathcal{L}_i}{\partial \theta_i^t}, \quad (22)$$

where η represents the learning rate and $\frac{\partial \mathcal{L}_i}{\partial \theta_i^t}$ is the gradient of the local model.

6 Security Analysis

In this section, we present a theoretical analysis demonstrating that P²FedRec can guarantee both data-level and edge-level privacy. It should be noted that in federated recommendation frameworks, users upload item embeddings rather than raw interaction data. However, existing studies have revealed that even uploading only item embeddings may still lead to raw data leakage. To address this, our approach achieves data-level privacy protection by safeguarding the privacy of item embeddings. Consequently, in the proofs presented in this section, data-level privacy is essentially equivalent to item embedding-level privacy.

6.1 Data-Level Privacy

THEOREM 6.1. *The data-level privacy holds against the users, if for vectors $\mathbf{x}$ and $\mathbf{y}$, their inner product $\mathbf{x} \cdot \mathbf{y}$ is computationally indistinguishable from $\mathbf{x} \cdot \mathbf{z}$, where $\mathbf{z}$ is a randomly selected vector.*

PROOF. For each user u_i , the servers provide the cosine similarity scores between their own item embeddings and those of all other users, i.e., $z_{i,j}^t, j \in [1, |\mathcal{U}|] \wedge j \neq i$, the average of these similarities, i.e., $\bar{z}^t$, user-specific aggregated item embeddings which represent the combined embeddings of their neighbors, i.e., $\hat{\mathbf{g}}_i^t$ and the global mean of all user-specific embeddings, i.e., $\hat{\mathbf{g}}_{average}^t$. Since the cosine similarity is based on the inner product of normalized high-dimensional item embeddings, even with knowledge of their own embeddings, user u_i cannot computationally distinguish the original embeddings of other users. Moreover, the user-specific embeddings combine information from multiple neighbors, typically a large number, making it impossible for u_i to extract any single neighbor's original embedding. Similarly, global aggregated statistics, including the average similarity score and global mean embedding contain information that has been sufficiently mixed across users, preventing u_i from inferring other users' original embeddings from these population-level measures. $\square$

THEOREM 6.2. *The data-level privacy holds against the servers if the additive homomorphic encryption is IND-CPA secure and the additive secret sharing is information-theoretic secure.*

PROOF. During the online phase, servers $\mathcal{P}_1$ and $\mathcal{P}_2$ initially receive their respective shares of item embeddings: $(\langle \mathbf{g}_i^t \rangle_1, \langle \hat{\mathbf{g}}_i^t \rangle_1)$ for $\mathcal{P}_1$ and $(\langle \mathbf{g}_i^t \rangle_2, \langle \hat{\mathbf{g}}_i^t \rangle_2)$ for $\mathcal{P}_2$. These shares are generated through a 2-out-of-2 ASS that provides information-theoretic security. This ensures that neither party can independently reconstruct the complete embeddings $\mathbf{g}_i^t$ and $\hat{\mathbf{g}}_i^t$ as long as they remain

non-colluding. To compute inner products between any $\bar{\mathbf{g}}_i^t$ and $\bar{\mathbf{g}}_j^t$ where $i, j \in [1, |\mathcal{U}|]$ and $j \neq i$, the servers employ ASS-based multiplication. In this process, they exchange perturbed shares $\langle \mathbf{e}_i^t \rangle_1 = \langle \bar{\mathbf{g}}_i^t \rangle_1 - \langle \mathbf{r}^{(i)} \rangle_1$ and $\langle \mathbf{e}_i^t \rangle_2 = \langle \bar{\mathbf{g}}_i^t \rangle_2 - \langle \mathbf{r}^{(i)} \rangle_2$ for all $i \in [1, |\mathcal{U}|]$, subsequently recovering $\mathbf{e}_i^t = \bar{\mathbf{g}}_i^t - \mathbf{r}^{(i)}$. The IND-CPA security of the additive homomorphic encryption guarantees that during Beaver triple preparation, $\mathcal{P}_1$ only accesses each element of $\langle \mathbf{r}^{(i)} \rangle_1$ (i.e., $\langle \mathbf{r}^{(i)} \rangle_1$) while $\mathcal{P}_2$ only accesses each element of $\langle \mathbf{r}^{(i)} \rangle_2$ (i.e., $\langle \mathbf{r}^{(i)} \rangle_2$). Consequently, $\mathcal{P}_1$ cannot deduce $\langle \bar{\mathbf{g}}_i^t \rangle_2$ from $\langle \mathbf{e}_i^t \rangle_2$, nor can $\mathcal{P}_2$ infer $\langle \bar{\mathbf{g}}_i^t \rangle_1$ from $\langle \mathbf{e}_i^t \rangle_1$, maintaining the confidentiality of the original $\bar{\mathbf{g}}_i^t, i \in [1, |\mathcal{U}|]$. Notably, in P²FedRec, the Beaver triples generated offline are consistently reused throughout all iteration rounds. Specifically, for each round $t \in [1, T]$, the normalized item embeddings $\bar{\mathbf{g}}_i^t$ are perturbed using the same predetermined vector $\mathbf{r}^{(i)}$. While servers can track the evolution of these perturbed embeddings across rounds, they cannot determine the exact embeddings at any round t , because the initial item embedding $\bar{\mathbf{g}}_i^1$ remains unknown to them. $\square$

6.2 Edge-Level Privacy

THEOREM 6.3. *The local graph holds edge-level privacy against the users by pseudo-anonymity.*

PROOF. Each user u_i constructs its local graph by connecting to neighbors with similar preferences. Through server-side pseudo-anonymization of all users, u_i can identify neighbors while remaining oblivious to their true identities, thereby guaranteeing identity anonymity. Since the server reapplies pseudo-anonymization in each round, u_i cannot determine whether any particular neighbor maintains persistent connections across rounds, which guarantees identity unlinkability. The above anonymity and unlinkability ensure the edge-level privacy in the local graph. Importantly, even when users collude with others, this edge-level privacy is still guaranteed. Although they can know collaborators' preferences, all other anonymous users still cannot be associated with real-world identities, thus preserving the privacy of non-colluding individuals' preferences. $\square$

THEOREM 6.4. *The noisy local graph holds ϵ -edge local differential privacy against the servers.*

PROOF. We use ϵ and γ to denote the privacy budget and degree privacy parameter, respectively. We let $l, l' \in \{0, 1\}^{|\mathcal{U}|}$ denote two adjacency lists that differ exclusively at the p -th bit, and $(a_1, a_2) \in \{0, 1\}^{|\mathcal{U}|} \times \mathbb{R}$ denote an arbitrary output of noise injection function used on the local graph in Sec. 5.2. For the sake of illustration, we denote the noise injection function as $\mathcal{N}$, the operation of adding Laplacian noise to degree d described by Eq. (9) as $\mathcal{A}$, the operation of random flipping described by Eq. (10) as $\mathcal{F}$. Assume $\mathcal{F}(l) = m, \mathcal{F}(l') = m', d = \sum_{i=1}^{|\mathcal{U}|} l_i, d' = \sum_{i=1}^{|\mathcal{U}|} l'_i$. Then, we have

$$\begin{aligned}
 \frac{\Pr[\mathcal{N}(l) = (a_1, a_2)]}{\Pr[\mathcal{N}(l') = (a_1, a_2)]} &= \frac{\Pr[\mathcal{F}(l) = a_1] \Pr[\mathcal{A}(d) = a_2]}{\Pr[\mathcal{F}(l') = a_1] \Pr[\mathcal{A}(d') = a_2]} \\
 &= \frac{\prod_{i=1}^{|\mathcal{U}|} \Pr[m_i = a_{1,i}]}{\prod_{i=1}^{|\mathcal{U}|} \Pr[m'_i = a_{1,i}]} \cdot \frac{\epsilon_d/2 \exp(-\epsilon_d|d - a_2|)}{\epsilon_d/2 \exp(-\epsilon_d|d' - a_2|)} \\
 &= \frac{\Pr[m_p = a_{1,p}]}{\Pr[m'_p = a_{1,p}]} \cdot \exp(-\epsilon_d(|d - a_2| - |d' - a_2|)) \\
 &\leq \frac{\exp(\epsilon_a)/(1 + \exp(\epsilon_a))}{1/(1 + \exp(\epsilon_a))} \cdot \exp(\epsilon_d|(d - a_2) - (d' - a_2)|) \\
 &= \exp(\epsilon_a) \cdot \exp(\epsilon_d) \\
 &= \exp(\epsilon_a + \epsilon_d) \\
 &= \exp(\epsilon)
 \end{aligned}$$

$\square$

Table 1. Dataset statistics

Dataset	Users	Items	Interactions
Movie-100K	943	1,682	100,000
Movie-1M	6,040	3,706	1,000,209
HetRec2011	2,113	10,109	855,598
Lastfm-2K	1,600	12,454	185,650
Douban	2,509	39,576	893,575
Beauty	22,363	12,101	198,502

REMARK 6.1. Note that in the execution process of P²FedRec, after receiving the noisy local graph, the servers perform appropriate denoising to reconstruct the adjacency matrix $\tilde{\mathcal{L}} \in \{0, 1\}^{|\mathcal{U}| \times |\mathcal{U}|}$ based on β -Model and Bayesian estimation. It is a common practice to allow servers to employ denoising techniques to recover meaningful aggregated statistics from perturbed user data. Such an operation does not violate privacy due to the post-processing property of LDP, as described in Theorem 2.1. Besides, Zhu et al. [51] have derived an upper bound for the estimation error between the estimated adjacency matrix $\tilde{\mathcal{L}}$ and the true graph topology $\mathcal{L}$, i.e., $E[\|\tilde{\mathcal{L}} - \mathcal{L}\|_{1,1}] \leq 2\|\mathcal{L}\|_{1,1} + \frac{|\mathcal{U}|}{2\epsilon_d}$, where $\|\mathcal{L}\|_{1,1} = \sum_{i=1}^{|\mathcal{U}|} \sum_{j=1}^{|\mathcal{U}|} |\mathcal{L}_{ij}|$, but empirical results indicate that this upper bound is not tight. Therefore, in Sec. 7, we will quantify the exact discrepancy between $\tilde{\mathcal{L}}$ and $\mathcal{L}$.

7 Experiment

In this section, we conduct extensive experiments to evaluate the performance of P²FedRec and answer the following series of questions.

- Q1: Does P²FedRec achieve a satisfactory balance between privacy protection and model performance?
- Q2: Does P²FedRec outperform the state-of-the-art federated recommendation works with only data-level privacy protection?
- Q3: How many computational and communication overheads does multi-level privacy protection in P²FedRec introduce for each entity?
- Q4: How do the key hyperparameters, including α , β , γ , and H affect P²FedRec?
- Q5: Can P²FedRec be extended to accommodate a larger-scale system characterized by an increased number of users?

7.1 Experimental Setting

7.1.1 Datasets. We first evaluate the performance and overhead of P²FedRec on five benchmark datasets [47], i.e., MovieLens-100K (denoted as Movie-100K), MovieLens-1M (denoted as Movie-1M), HetRec2011, Lastfm-2K, and Douban. Then, we discuss the scalability of P²FedRec and conduct validation using a larger-scale dataset, i.e., Beauty [13]. The two MovieLens datasets are sourced from the MovieLens website, containing user ratings for films with a minimum of 20 ratings per user. HetRec2011 incorporates additional web-based information by linking movies to their corresponding pages on both Internet Movie Database and Rotten Tomatoes review platforms. Lastfm-2K contains music listening records, including artist lists and listening counts per user. We filtered out users with fewer than five recorded interactions. Douban is also a movie dataset. Beauty is a user-clothing interaction dataset collected from the Amazon website. The detailed statistical characteristics of these datasets are summarized in Table 1.

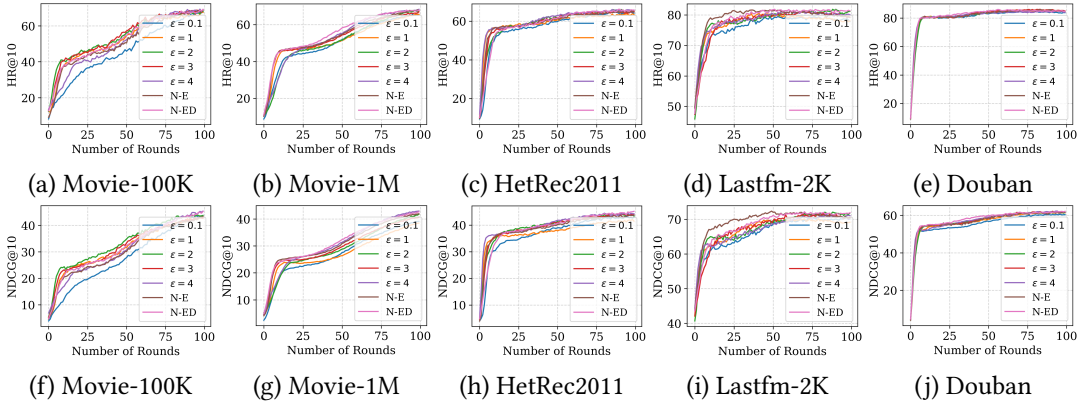


Fig. 3. Model convergence comparison on five datasets, where N-E and N-ED denote P²FedRec without considering edge-level privacy, and without considering edge-level and data-level privacy, respectively.

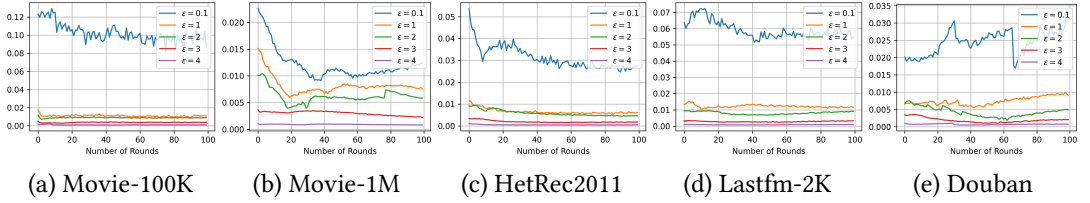


Fig. 4. Mean absolute error (MAE) of reconstructed adjacency matrix $\hat{\mathcal{L}}$ against ground truth $\mathcal{L}$.

7.1.2 Evaluation metrics. For fair experimental evaluation, we employ a widely-used leave-one-out setting [47]. To be specific, we split each user's interactions by reserving their most recent item as the test sample and using the remaining items for training. Following common practice [47], we optimize computational efficiency during evaluation by ranking the test item alongside 99 randomly sampled negative items instead of evaluating against the entire item pool. Then, HR@10 (Hit Ratio) and NDCG@10 (Normalized Discounted Cumulative Gain) are used to measure the model performance. The former evaluates a system's capability to identify relevant items within the top-10 recommendations, while the latter additionally examines the positional distribution of relevant items by implementing a monotonically decreasing weighting scheme that assigns greater importance to higher-ranked items.

7.1.3 Baselines. The baselines employed in this paper are described as follows:

- **FedMF** [6]: It is a generic FedRec, where each user locally updates user embeddings and encrypts the gradients of these embeddings using HE before uploading them to the server for global aggregation.
- **PFedRec** [46]: It is a personalized FedRec, where each perturbs item embeddings using LDP before uploading them to the server for aggregation, and then locally fine-tunes the aggregated embeddings to obtain personalized models.
- **GPFedRec** [47]: It is also a personalized FedRec, which differentiates from PFedRec [46] by incorporating user relationship graphs into the modeling process.

7.1.4 Implementation details. In our experiments, the precision Q is set to 15, the ring is chosen as $\mathbb{Z}_{2^{32}}$. For every positive sample, our approach selects four unobserved items at random to serve as

Table 2. Performance comparison on five datasets. PFedRec_s and PFedRec_w denote PFedRec with strong (i.e., $\epsilon = 1$) and weak (i.e., $\epsilon = 5$) level data privacy, GPFedRec_s and GPFedRec_w denote GPFedRec with strong (i.e., $\epsilon = 1$) and weak (i.e., $\epsilon = 5$) level data privacy, Li_P²FedRec denotes a lightweight variant of P²FedRec by reducing the frequency of relationship graph updates, HR and NDCG denote HR@10 and NDCG@10, respectively, D_P and DE_P denote the schemes providing data-level privacy, and providing data-level and edge-level privacy, respectively.

	Method	Movie-100K		Movie-1M		HetRec2011		Lastfm-2K		Douban	
		HR	NDCG	HR	NDCG	HR	NDCG	HR	NDCG	HR	NDCG
D_P	FedMF	65.67	37.84	65.94	38.73	62.17	38.36	79.72	68.68	83.89	61.12
	PFedRec_s	69.17	40.52	67.21	39.19	63.61	38.27	80.04	70.24	83.31	59.22
	PFedRec_w	70.12	41.58	68.11	40.25	64.20	39.24	81.23	71.52	84.03	61.25
	GPFedRec_s	68.29	38.95	66.57	38.86	63.22	38.16	80.12	70.43	83.65	59.64
	GPFedRec_w	69.03	41.11	67.81	40.09	64.72	39.82	81.03	71.65	84.21	61.95
	N-E	<u>71.37</u>	<u>42.66</u>	<u>68.36</u>	<u>40.95</u>	<u>65.45</u>	<u>39.95</u>	<u>81.24</u>	<u>71.87</u>	<u>85.00</u>	<u>61.79</u>
DE_P	P ² FedRec	71.34	42.64	68.01	40.21	65.69	40.55	81.19	71.79	84.93	61.76
	Li_P ² FedRec	70.89	41.57	67.59	40.02	65.21	40.31	81.14	71.77	84.73	62.10

negative training examples to construct training data, a strategy consistently applied across all compared approaches. To ensure a fair comparison, we keep all hyperparameters in the baseline methods identical to those reported in their original papers. We set the batch size to 256 and the learning rate to 0.1. All approaches are trained for 100 epochs to ensure stable convergence. Besides, the prototype of P²FedRec is implemented in Python, and runs on the machine with Intel Xeon Gold 6430 @ 2.40GHz, RTX 4090 GPU.

7.2 Performance analysis(Q1+Q2)

Fig. 3 and Table 2³ illustrate the recommendation performance of P²FedRec on five benchmark datasets. Subsequently, we will consolidate the experimental results and analyze two pivotal findings.

(1) P²FedRec achieves a satisfactory privacy-utility trade-off across five datasets. We compare six P²FedRec variants against the conventional federated recommendation without considering any privacy (referred to as N-ED), including a base version (N-E) offering only data-level privacy protection and five enhanced variants that additionally provide edge-level privacy protection at different levels ($\epsilon = 0.1, 1, 2, 3, 4$). As shown in Fig. 3, the convergence speed of the model decreases with the reduction of privacy budget across all datasets (e.g., reaching the slowest convergence when the privacy budget is set to 0.1). This phenomenon is particularly pronounced on the MovieLens-100K, which contains the smallest number of users. However, when the iteration round increases to 100, almost all P²FedRec with different ϵ achieve performance comparable to the conventional method (i.e., N-ED), which can be attributed to the noise dilution achieved by the density of the relationship graph and the denoising mechanism used. To further quantify the utility of this mechanism, Fig. 4 presents the mean absolute error (MAE, computed as $\frac{1}{|\mathcal{U}|^2} \sum_{i,j} |\hat{\mathcal{L}}_{ij} - \mathcal{L}_{ij}|$) between the estimated adjacency matrix $\hat{\mathcal{L}}$ and the ground-truth matrix $\mathcal{L}$ under varying privacy budgets. The results demonstrate that the MAE between $\hat{\mathcal{L}}$ and $\mathcal{L}$ remains consistently low across all datasets.

³All reported HR@10 and NDCG@10 scores for the baselines were obtained under our experimental setup; due to differences in experimental environments, there may be minor deviations from the original papers.

Table 3. Computational and communication overheads of three building components, where Cons., Aggr., and Upda. represent embedding-shared local graph construction, noisy global graph-guided aggregation and personalized model updating, respectively, Comp. and Comm. represent computational overhead and communication overhead, respectively.

Phase		Movie-100K		Movie-1M		HetRec2011		Lastfm-2K		Douban	
		Comp.	Comm.	Comp.	Comm.	Comp.	Comm.	Comp.	Comm.	Comp.	Comm.
Cons.	$\mathcal{P}_1$	0.12s	0.76GB	5.57s	10.81GB	1.80s	10.20GB	1.65s	9.51GB	10.17s	47.37GB
	$\mathcal{P}_2$	0.001s	0.76GB	0.10s	10.81GB	0.01s	10.20GB	0.01s	9.51GB	0.01s	47.37GB
	User	0.18s	0.82MB	0.48s	1.83MB	0.07s	4.94MB	0.06s	6.09MB	0.11s	19.33MB
Aggr.	$\mathcal{P}_1$	1.84s	0.38GB	64.73s	5.37GB	12.83s	5.10GB	10.32s	4.75GB	0.11s	23.68GB
	$\mathcal{P}_2$	1.72s	0.38GB	65.34s	5.37GB	12.60s	5.10GB	10.30s	4.75GB	36.03s	23.68GB
	User	—	0.82MB	—	1.82MB	—	4.94MB	—	6.08MB	—	19.33MB
Upda.	$\mathcal{P}_1/\mathcal{P}_2$	—	—	—	—	—	—	—	—	—	—
	User	0.16s	—	0.17s	—	0.17s	—	0.20s	—	0.25s	—

Table 4. Computational overhead comparison per round.

Datasets	FedMF	PFedRec	GPFedRec	P ² FedRec
Movie-100K	8307.57s	11.84s	17.51s	18.68s
Movie-1M	16304.32s	151.36s	204.24s	325.76s
HetRec2011	48929.40s	206.73s	222.80s	257.25s
Lastfm-2K	59511.60s	54.20s	85.62s	89.52s
Douban	175470.21s	686.22s	779.16s	881.26s

(2) **P²FedRec outperforms all baseline methods across five datasets.** Existing methods (e.g., PFedRec and GPFedRec) primarily employ LDP to achieve data-level privacy protection, which inevitably compromises model performance while enhancing privacy guarantees. For instance, GPFedRec_s ($\epsilon = 1$) exhibits a reduction of 1.5 in HR@10 and 1.66 in NDCG@10 on HetRec2011 compared to GPFedRec_w ($\epsilon = 5$). In contrast, we employ ASS to protect data-level privacy, effectively circumventing the privacy-efficiency trade-off dilemma faced by existing approaches. Besides, as analyzed above, applying LDP to the relationship graph can achieve a satisfactory privacy-utility trade-off. Consequently, incorporating edge-level privacy preservation does not significantly degrade model performance in P²FedRec. For instance, on the MovieLens-100K, while N-E achieves an HR@10 of 71.37 and NDCG@10 of 42.66, P²FedRec maintains competitive results of 71.34 (HR@10) and 42.64 (NDCG@10).

7.3 Overhead analysis (Q3)

Table 3 summarizes the computational and communication overheads of the three building components of P²FedRec. It can be observed that P²FedRec achieves minimal computational and communication overhead at the user side, making it ideally suited for resource-constrained user devices in federated recommendation scenarios. Notably, in Cons., a significant portion of the server-side communication overhead originates from exchanging masked item embeddings when computing similarity shares. Since the user relationship graph (derived from historical preference similarity) exhibits temporal stability, we can use a periodic graph updating strategy to reduce

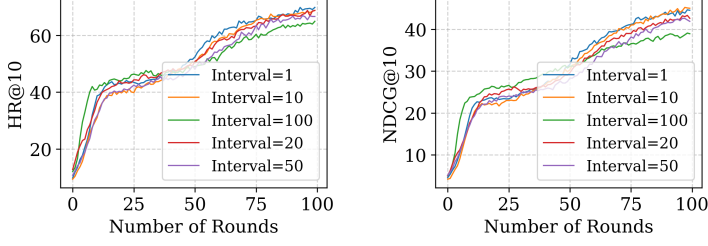


Fig. 5. Impact of Graph Update Frequency on Model Convergence on the dataset MovieLens-100K.

server-side communication overhead, and validate the efficacy of this strategy on the dataset MovieLens-100K, as depicted in Fig. 5. The experimental results demonstrate that selecting an appropriate update interval (e.g., 10 or 20) can achieve an excellent efficiency-performance balance. Table 4 summarizes the comparison of its per-round computational overhead with existing methods. We can observe that the per-round computational overhead of P²FedRec is comparable to existing LDP-based approaches. This is because we offload the costly preparation of Beaver triples to the offline phase.

7.4 Hyperparameter analysis (Q4)

In this part, we conduct experiments to analyze the impact of several key hyperparameters on model performance.

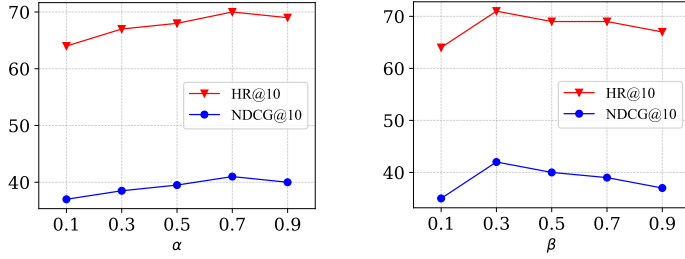


Fig. 6. Effect of key hyperparameters α and β on the dataset Movie-100K.

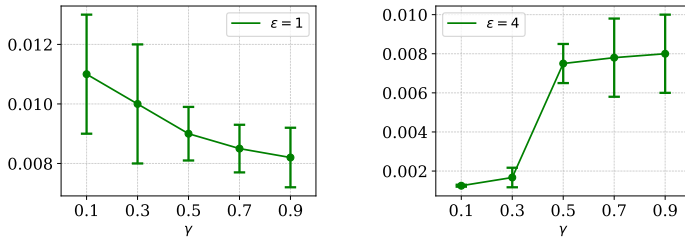


Fig. 7. Effect of key hyperparameter γ on the dataset Movie-100K.

The threshold factor α for selecting neighbors. We fix all other parameters while varying threshold α from 0.1 to 0.9 in steps of 0.2. As illustrated in Fig. 6, we can observe that

(1) The model achieves optimal recommendation performance when α is set to 0.7, with observable performance degradation occurring at both higher and lower values.

(2) When α is set to 0.1, the relationship graph becomes almost fully connected. In this regime, the server-aggregated user-specific item embeddings collapse to the global average of all item embeddings, diminishing their capacity to capture individual preferences.

(3) When α is increased, for example, to 0.9, the number of neighbors for each node in the user relationship graph will be significantly reduced. Under such circumstances, the personalized item embeddings generated by the server for users will be biased and fail to accurately reflect users' preferences, thereby affecting the overall performance of the model and causing it to decline.

The regularization coefficient β . We fix all other parameters while systematically varying β from 0.1 to 0.9 in 0.2 increments. The experimental results presented in Fig. 6 demonstrate that

(1) As the β increases incrementally, the model performance first improves and then declines, achieving the best results when the coefficient is set to 0.3.

(2) Initially, as the regularization coefficient increases, the model performance improves progressively. This is because the increased weight of the regularization term enables the model to focus more effectively on capturing user-specific preference characteristics during training.

(3) Setting the regularization coefficient too high can degrade model performance. The regularization term aims to guide the model to leverage users with similar preferences, while the loss function focuses on capturing user-specific features from local data. However, an excessively large coefficient overemphasizes user similarity, causing users to deviate from their true preferences and disrupting the model training process.

The factor γ for Privacy Allocation. To balance privacy and utility in edge-level protection, we use γ to dynamically allocate privacy budgets between the adjacency matrix and node degrees. As shown in Fig. 7, we can see that

(1) The impact of γ on server-side adjacency matrix reconstruction exhibits significant variation across different ϵ .

(2) When ϵ is small (i.e., $\epsilon = 1$), the random response mechanism may introduce excessive noise, distorting the adjacency matrix. Under such circumstances, the prior probability calculated based on the noise degree sequence will become a key element for the server to denoise the matrix. Notably, increasing γ (i.e., allocating more budget to degree information) at this point can reduce the error between the estimated and the true graphs, yet this slight change in estimation error has almost no effect on model performance. The reason is that the relationship graph is inherently dense, so these tiny perturbations are too small to alter its overall structure.

(3) When ϵ is high (i.e., $\epsilon = 4$), using a smaller γ can surprisingly yield better performance. This occurs because the perturbation probability in randomized response decreases significantly, making the noisy adjacency matrix itself sufficiently informative. Therefore, in this scenario, more privacy budget should be allocated to the adjacency list.

The size of embedding H . We evaluate how user and item embedding sizes affect model performance. The results are depicted in Fig. 8, where the embedding size is set from 16 to 128. It can be observed that the model achieves optimal performance when the embedding size is set to 32. The reason is that when the embedding dimension is set too small, the model suffers from insufficient capacity, leading to significantly worse performance compared to other configurations. Conversely, an excessively large dimension induces overfitting, which similarly degrades model performance.

7.5 Scalability Discussion (Q5)

In practical applications, when recommendation systems serve a large number of users, constructing a global user relationship graph on the server side faces scalability challenges. Considering that

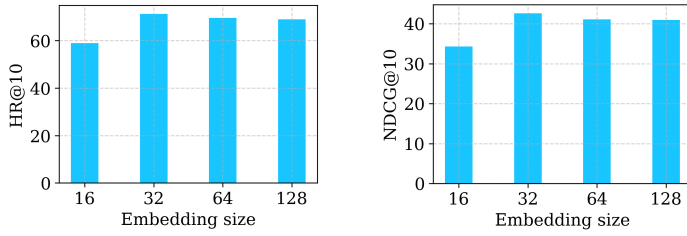


Fig. 8. Impact of embedding size on MovieLens-100K.

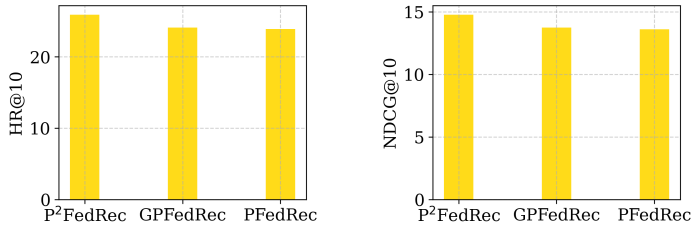


Fig. 9. Performance comparison on the dataset Beauty.

when the user base expands to a larger scale, the overlap in user interests increases, resulting in data redundancy. At this point, it is unnecessary to construct a complete graph. Thus, we can employ an optimized strategy: dividing the large-scale users into multiple subsets and independently constructing subgraphs for each subset. To verify this strategy, we select a larger-scale real-world dataset Beauty [13], and randomly divide it into four groups in each round to construct the corresponding relationship graphs. The performance of P²FedRec can be seen in Fig. 9, where P²FedRec remains effective. This demonstrates that P²FedRec can be scaled to larger systems via the above strategy.

8 Related Work

8.1 Federated Recommendation

Federated recommendation [2, 32, 43, 48], as an emerging frontier, aims to deliver services via federated learning (FL) [4, 12, 36] framework while safeguarding user privacy. A variety of recommendation algorithms have been successfully adapted to the FL framework, including FCF [3], MetaMF [23], and FedRecon [34] based on matrix factorization, as well as FedNCF [31] based on neural collaborative filtering. Besides, Liu *et al.* [25] enhanced the local user-item interaction graph by incorporating user social relationship information. However, in practical applications, a publicly accessible social network is not always available. Wu *et al.* [40] proposed FedPerGNN, which leveraged graph neural networks to capture the correlations in local user-item interactions and utilized a third-party server to identify higher-order neighbors to optimize local model training. In reality, the third-party server is often not fully trustworthy. Thus, aligning historical interactions by third-party servers inevitably increases the computational burden and poses the risk of user privacy leakage. Qu *et al.* [32] noted that some users, less concerned with privacy, were willing to upload personal data for better recommendations. Therefore, they proposed a cloud-device collaborative graph neural network federated recommendation model named CDCGNNFed, which significantly

enhanced the practicality of the federated recommendation system through a user-led data contribution mechanism and a graph mending strategy. Recently, Zhang *et al.* [47] proposed a graph-guided personalized federated recommendation method that constructed a user-relation graph based on users' historical item interaction records. Although it achieved significant improvements, it directly exposed the item embeddings and user relationships to the untrusted server.

8.2 Privacy-enhanced Federated Recommendation

Recently, Chai *et al.* [6] have demonstrated that although users in federated recommendation only upload gradient information to the server, it is still possible for the original interaction data of users to be leaked. Hence, a prominent research avenue [8, 9, 16, 24, 28, 30, 35, 43] centers on integrating data perturbation and cryptographic techniques into the federated recommendation framework, striving to bolster data security while ensuring accurate recommendations. For example, Minto *et al.* [28] proposed a practical federated recommendation system that processed implicit data under the framework of user-level local differential privacy (LDP). By introducing a proxy network, the system anonymizes and randomizes reports before forwarding them to the recommendation system, effectively reducing the risk of fingerprint identification and thereby further enhancing user privacy protection. However, the introduction of differential privacy inevitably affects the accuracy of the model. Nguyen *et al.* [30] encrypted the gradients using homomorphic encryption before uploading them, and then the server can performed aggregation operations on the encrypted data. Although the use of homomorphic encryption does not sacrifice the precision of the model, the frequent encryption and decryption operations, as well as the transmission of ciphertexts, can impose a significant computational and communication burden on the user side. Besides, Lin *et al.* [24] designed an effective and generic federated recommendation framework based on fake marks and secret sharing, which could effectively protect the privacy of rating values and rating behaviors. However, the aforementioned privacy-enhancing federated recommendation work has focused solely on the privacy of users' local data, while overlooking that the relationships between users are also privacy-sensitive.

9 Conclusion

In this paper, we propose P²FedRec, a personalized federated recommendation framework that addresses multi-level privacy concerns while effectively capturing user-specific preferences. P²FedRec uses a two-server model and customized interaction protocol to protect data-level and edge-level privacy, overcoming the limitations of conventional one-size-fits-all federated recommendation systems. To ensure data-level privacy, we introduce an embedding-shared local graph construction component that enables the construction of user relationship graphs without exposing user data. We then capture user-specific preferences through a noisy global graph-guided aggregation component while protecting relationship privacy. Finally, a tailored training component dynamically adapts to individual user preferences. Rigorous theoretical analysis and extensive experiments on five real-world datasets demonstrate that P²FedRec outperforms the state-of-the-art solutions while ensuring data-level and edge-level privacy.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (Grant No. 62472032), the Young Elite Scientists Sponsorship Program by CAST (Grant No. 2023QNRC001), the National Science and Technology Major Project (Grant No. 2025ZD0620000), Key Laboratory of Industrial Data Security and Protection Testing, Ministry of Industry and Information Technology Open Research Project (Grant No. IDPSA2025-004), and partially sponsored by CAAI-CANN Open Fund, developed on OpenI Community.

References

- [1] Abbas Acar, Hidayet Aksu, A Selcuk Ulugac, and Mauro Conti. 2018. A survey on homomorphic encryption schemes: Theory and implementation. *ACM Computing Surveys (Csur)* 51, 4 (2018), 1–35.
- [2] Nimesh Agrawal, Anuj Kumar Sirohi, Sandeep Kumar, et al. 2024. No prejudice! fair federated graph neural networks for personalized recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 10775–10783.
- [3] Muhammad Ammad-Ud-Din, Elena Ivannikova, Suleiman A Khan, Were Oyomno, Qiang Fu, Kuan Eeik Tan, and Adrian Flanagan. 2019. Federated collaborative filtering for privacy-preserving personalized recommendation system. *arXiv preprint arXiv:1901.09888* (2019).
- [4] Alia Asheralieva, Dusit Niyato, and Xuetao Wei. 2025. Dynamic Distributed Model Compression for Efficient Decentralized Federated Learning and Incentive Provisioning in Edge Computing Networks. *IEEE Transactions on Mobile Computing* (2025). doi:10.1109/TMC.2025.3543295
- [5] Luciano Caroprese, Francesco Sergio Pisani, Bruno Miguel Veloso, Matthias Konig, Giuseppe Manco, Holger Hoos, and Joao Gama. 2025. Modelling Concept Drift in Dynamic Data Streams for Recommender Systems. *ACM Transactions on Recommender Systems* 3, 2 (2025), 1–28.
- [6] Di Chai, Leye Wang, Kai Chen, and Qiang Yang. 2020. Secure federated matrix factorization. *IEEE Intelligent Systems* 36, 5 (2020), 11–20.
- [7] Sourav Chatterjee, Persi Diaconis, and Allan Sly. 2011. Random graphs with a given degree sequence. (2011).
- [8] Gaode Chen, Xinghua Zhang, Yijun Su, Yantong Lai, Ji Xiang, Junbo Zhang, and Yu Zheng. 2023. Win-win: a privacy-preserving federated framework for dual-target cross-domain recommendation. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 4149–4156.
- [9] Qian Chen, Zilong Wang, Mengqing Yan, Haonan Yan, Xiaodong Lin, and Jianying Zhou. 2025. DeFedGCN: Privacy-Preserving Decentralized Federated GCN for Recommender System. *IEEE Transactions on Services Computing* (2025). doi:10.1109/TSC.2025.3536320
- [10] Ivan Damgård, Hannah Keller, Boel Nelson, Claudio Orlandi, and Rasmus Pagh. 2024. Differentially private selection from secure distributed computing. In *Proceedings of the ACM Web Conference 2024*. 1103–1114.
- [11] Yucheng Ding, Chaoyue Niu, Fan Wu, Shaojie Tang, Chengfei Lyu, and Guihai Chen. 2024. Enhancing on-device llm inference with historical cloud-based llm interactions. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 597–608.
- [12] Chao hao Fu, Hongbin Chen, and Na Ruan. 2025. Privacy for Free: Spy Attack in Vertical Federated Learning by Both Active and Passive Parties. *IEEE Transactions on Information Forensics and Security* 20 (2025), 2550–2563.
- [13] Mingzhe Han, Dongsheng Li, Jiafeng Xia, Jiahao Liu, Hansu Gu, Peng Zhang, Ning Gu, and Tun Lu. 2025. FedCIA: Federated Collaborative Information Aggregation for Privacy-Preserving Recommendation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1687–1696.
- [14] Alexandra Henzinger, Matthew M Hong, Henry Corrigan-Gibbs, Sarah Meiklejohn, and Vinod Vaikuntanathan. 2023. One server for the price of two: Simple and fast {Single-Server} private information retrieval. In *32nd USENIX Security Symposium (USENIX Security 23)*. 3889–3905.
- [15] Chenfei Hu, Chuan Zhang, Dian Lei, Tong Wu, Ximeng Liu, and Liehuang Zhu. 2023. Achieving privacy-preserving and verifiable support vector machine training in the cloud. *IEEE Transactions on Information Forensics and Security* 18 (2023), 3476–3491.
- [16] Zhicong Huang, Cheng Hong, Chenkai Weng, Wen-jie Lu, and Hunter Qu. 2021. More efficient secure matrix multiplication for unbalanced recommender systems. *IEEE Transactions on Dependable and Secure Computing* 20, 1 (2021), 551–562.
- [17] Thanh Trung Huynh, Trong Bang Nguyen, Thanh Toan Nguyen, Phi Le Nguyen, Hongzhi Yin, Quoc Viet Hung Nguyen, and Thanh Tam Nguyen. 2025. Certified unlearning for federated recommendation. *ACM Transactions on Information Systems* 43, 2 (2025), 1–29.
- [18] Jacob Imola, Takao Murakami, and Kamalika Chaudhuri. 2021. Locally differentially private analysis of graph statistics. In *30th USENIX security symposium*. 983–1000.
- [19] Gyuseok Lee, SeongKu Kang, Wonbin Kweon, and Hwanjo Yu. 2024. Continual Collaborative Distillation for Recommender System. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 1495–1505.
- [20] Dian Lei, Zijun Song, Yanli Yuan, Chunhai Li, and Liehuang Zhu. 2025. Achieving Personalized Privacy-Preserving Graph Neural Network via Topology Awareness. In *Proceedings of the ACM on Web Conference 2025*. 3552–3560.
- [21] Xueyang Li, Xue Yang, Zhengchun Zhou, and Rongxing Lu. 2024. Efficiently achieving privacy preservation and poisoning attack resistance in federated learning. *IEEE Transactions on Information Forensics and Security* 19 (2024), 4358–4373.
- [22] Zhiwei Li, Guodong Long, Tianyi Zhou, Jing Jiang, and Chengqi Zhang. 2025. Personalized federated collaborative filtering: A variational autoencoder approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39.

- 18602–18610.
- [23] Yujie Lin, Pengjie Ren, Zhumin Chen, Zhaochun Ren, Dongxiao Yu, Jun Ma, Maarten de Rijke, and Xiuzhen Cheng. 2020. Meta matrix factorization for federated rating predictions. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 981–990.
 - [24] Zhaohao Lin, Weike Pan, Qiang Yang, and Zhong Ming. 2022. A generic federated recommendation framework via fake marks and secret sharing. *ACM Transactions on Information Systems* 41, 2 (2022), 1–37.
 - [25] Zhiwei Liu, Liangwei Yang, Ziwei Fan, Hao Peng, and Philip S Yu. 2022. Federated social recommendation with graph neural network. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 4 (2022), 1–24.
 - [26] Sichun Luo, Yuanzhang Xiao, and Linqi Song. 2022. Personalized federated recommendation via joint representation learning, user clustering, and model adaptation. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 4289–4293.
 - [27] Xingyuan Mao, Yuwen Liu, Lianying Qi, Li Duan, Xiaolong Xu, Xuyun Zhang, Wanchun Dou, Amin Beheshti, and Xiaokang Zhou. 2024. Cluster-driven personalized federated recommendation with interest-aware graph convolution network for multimedia. In *Proceedings of the 32nd ACM International Conference on Multimedia*. 5614–5622.
 - [28] Lorenzo Minto, Moritz Haller, Benjamin Livshits, and Hamed Haddadi. 2021. Stronger privacy for federated collaborative filtering with implicit feedback. In *Proceedings of the 15th ACM conference on recommender systems*. 342–350.
 - [29] Payman Mohassel and Yupeng Zhang. 2017. Secureml: A system for scalable privacy-preserving machine learning. In *IEEE symposium on security and privacy*. 19–38.
 - [30] Ngoc-Hieu Nguyen, Tuan-Anh Nguyen, Tuan Nguyen, Vu Tien Hoang, Dung D Le, and Kok-Seng Wong. 2024. Towards Efficient Communication and Secure Federated Recommendation System via Low-rank Training. In *Proceedings of the ACM Web Conference 2024*. 3940–3951.
 - [31] Vasileios Perifanis and Pavlos S Efrimidis. 2022. Federated neural collaborative filtering. *Knowledge-Based Systems* 242 (2022), 108441.
 - [32] Liang Qu, Wei Yuan, Ruiqi Zheng, Lizhen Cui, Yuhui Shi, and Hongzhi Yin. 2024. Towards personalized privacy: User-governed data contribution for federated recommendation. In *Proceedings of the ACM Web Conference 2024*. 3910–3918.
 - [33] Pengcheng Ren, Chaoshun Zuo, Xiaofeng Liu, Wenrui Diao, Qingchuan Zhao, and Shanqing Guo. 2024. Demistify: Identifying on-device machine learning models stealing and reuse vulnerabilities in mobile apps. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
 - [34] Karan Singhal, Hakim Sidahmed, Zachary Garrett, Shanshan Wu, John Rush, and Sushant Prakash. 2021. Federated reconstruction: Partially local federated learning. *Advances in Neural Information Processing Systems* 34 (2021), 11220–11232.
 - [35] Changxin Tian, Yuexiang Xie, Xu Chen, Yaliang Li, and Xin Zhao. 2024. Privacy-preserving cross-domain recommendation with federated graph learning. *ACM Transactions on Information Systems* 42, 5 (2024), 1–29.
 - [36] Shuo Wang, Keke Gai, Jing Yu, Zijian Zhang, and Liehuang Zhu. 2025. Pravfed: Practical heterogeneous vertical federated learning via representation learning. *IEEE Transactions on Information Forensics and Security* 20 (2025), 2693–2705.
 - [37] Stanley L Warner. 1965. Randomized response: A survey technique for eliminating evasive answer bias. *Journal of the American statistical association* 60, 309 (1965), 63–69.
 - [38] Kang Wei, Jun Li, Ming Ding, Chuan Ma, Howard H Yang, Farhad Farokhi, Shi Jin, Tony QS Quek, and H Vincent Poor. 2020. Federated learning with differential privacy: Algorithms and performance analysis. *IEEE transactions on information forensics and security* 15 (2020), 3454–3469.
 - [39] Shihong Wei, Zhiwei Wang, Xubin An, Qian Li, Hanchun Xiao, and Yunpeng Xiao. 2024. A recommendation model for e-commerce platforms oriented to explicit information compensation and hidden information mining. *Knowledge-Based Systems* 286 (2024), 111359.
 - [40] Chuhan Wu, Fangzhao Wu, Lingjuan Lyu, Tao Qi, Yongfeng Huang, and Xing Xie. 2022. A federated graph neural network framework for privacy-preserving personalization. *Nature Communications* 13, 1 (2022), 3091.
 - [41] Chuhan Wu, Fangzhao Wu, Tao Qi, Yongfeng Huang, and Xing Xie. 2022. FedAttack: Effective and covert poisoning attack on federated recommendation via hard sampling. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4164–4172.
 - [42] Jianhang Xie, Chuntao Ding, Qingji Guan, Ao Zhou, and Yidong Li. 2025. ReFrame: A Resource-Friendly Cloud-Assisted On-Device Deep Learning Framework for Vision Services. *IEEE Transactions on Services Computing* (2025). doi:10.1109/TSC.2025.3552328
 - [43] Bo Yan, Yang Cao, Haoyu Wang, Wenchuan Yang, Junping Du, and Chuan Shi. 2024. Federated heterogeneous graph neural network for privacy-preserving recommendation. In *Proceedings of the ACM Web Conference 2024*. 3919–3929.
 - [44] Q Ye, H Hu, MH Au, X Meng, X Xiao, et al. 2022. LF-GDPR: a framework for estimating graph metrics with local differential privacy. *IEEE transactions on knowledge and data engineering* 34, 10 (2022), 4905–4920.

- [45] Junliang Yu, Hongzhi Yin, Xin Xia, Tong Chen, Jundong Li, and Zi Huang. 2023. Self-supervised learning for recommender systems: A survey. *IEEE Transactions on Knowledge and Data Engineering* 36, 1 (2023), 335–355.
- [46] Chunxu Zhang, Guodong Long, Tianyi Zhou, Peng Yan, Zijian Zhang, Chengqi Zhang, and Bo Yang. 2023. Dual personalization on federated recommendation. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 4558–4566.
- [47] Chunxu Zhang, Guodong Long, Tianyi Zhou, Zijian Zhang, Peng Yan, and Bo Yang. 2024. GPFedRec: Graph-guided personalization for federated recommendation. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 4131–4142.
- [48] Chunxu Zhang, Guodong Long, Tianyi Zhou, Zijian Zhang, Peng Yan, and Bo Yang. 2024. When federated recommendation meets cold-start problem: Separating item attributes and user interactions. In *Proceedings of the ACM Web Conference 2024*. 3632–3642.
- [49] Shijie Zhang, Wei Yuan, and Hongzhi Yin. 2023. Comprehensive privacy analysis on federated recommender system against attribute inference attacks. *IEEE Transactions on Knowledge and Data Engineering* 36, 3 (2023), 987–999.
- [50] Yuemin Zhang, Qingqing Ye, and Haibo Hu. 2025. Federated Heavy Hitter Analytics with Local Differential Privacy. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.
- [51] Xiaochen Zhu, Vincent YF Tan, and Xiaokui Xiao. 2023. Blink: link local differential privacy in graph neural networks via Bayesian estimation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. 2651–2664.

Received April 2025; revised July 2025; accepted August 2025